

Arduino チュートリアル

内容目次

1. プログラミング学習	2
開始前に：重要な注意事項	2
1.1 Arduino IDE をインストールする	2
1.2 Arduino IDE 用の ESP32 開発環境をインストールする	4
1.3 ライブラリファイルをインストールする	7
1.4 CH340 USB ドライバをインストールする	8
Windows システムにドライバーをインストールする	9
Linux システムにドライバーをインストールする	9
MAC システムにドライバーをインストールする	10
1.5 コード例	11
1.5.1 オンボードブザー	11
1.5.2 サーボを駆動する	14
1.5.3 ジョイスティックの値を読み取る	16
1.5.4 ジョイスティックで eArm を制御する(記録機能付き)	17
1.5.5 Web アプリの値を読み取る	20
1.5.6 ロボットアームの使用	23
2. ファームウェアの書き込み	24
2.1 書き込みツール	25
2.2 eArm ファームウェア（工場出荷時ファームウェア）	25
2.2.1 eArm ファームウェアの書き込み	26
2.3 ジョイスティック制御ファームウェア（記録機能付き）	27
2.3.1 ジョイスティック制御ファームウェアの書き込み	28
3. QA	30
3.1 USB シリアルポートが認識されない場合	30
3.2 ロボットアームが動作しない場合	30
3.3 その他の問題	30
4. お問い合わせ	30

プログラミング学習

開始前に：重要な注意事項

ESP32 コントローラーボードには、あらかじめロボットアームを操作するためのファームウェアが書き込まれています。このファームウェアにより、「組立・取扱説明書」に記載されているロボットアームの機能をご利用いただけます。書き込まれたファームウェアは、新しいプログラムを書き込むまで保持されます。

プログラムを書き込む前にご確認ください：

- 1: ESP32 ボードは、最後に書き込まれたプログラムを実行します
- 2: 新しいプログラムを書き込むと、それまでのプログラムは上書きされます

このセクションでは、基本から高度なコード例までを通じてロボットのプログラミングを解この章では、基本的なサンプルから応用的なサンプルまで、段階的にロボットのプログラミング方法をご紹介します。最後のサンプルには、工場出荷時にあらかじめ書き込まれているファームウェアのソースコードが含まれており、いつでもロボットアームを初期状態の動作に復元することができます。

1.1 Arduino IDE をインストールする

Arduino IDE の最新バージョンは Arduino 公式ウェブサイトからダウンロードできます：

<https://www.arduino.cc/en/software>

以下では、Arduino IDE (Windows Win 10 and newer, 64 bits) のインストールについて説明します。



Arduino IDE 2.3.2

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

SOURCE CODE

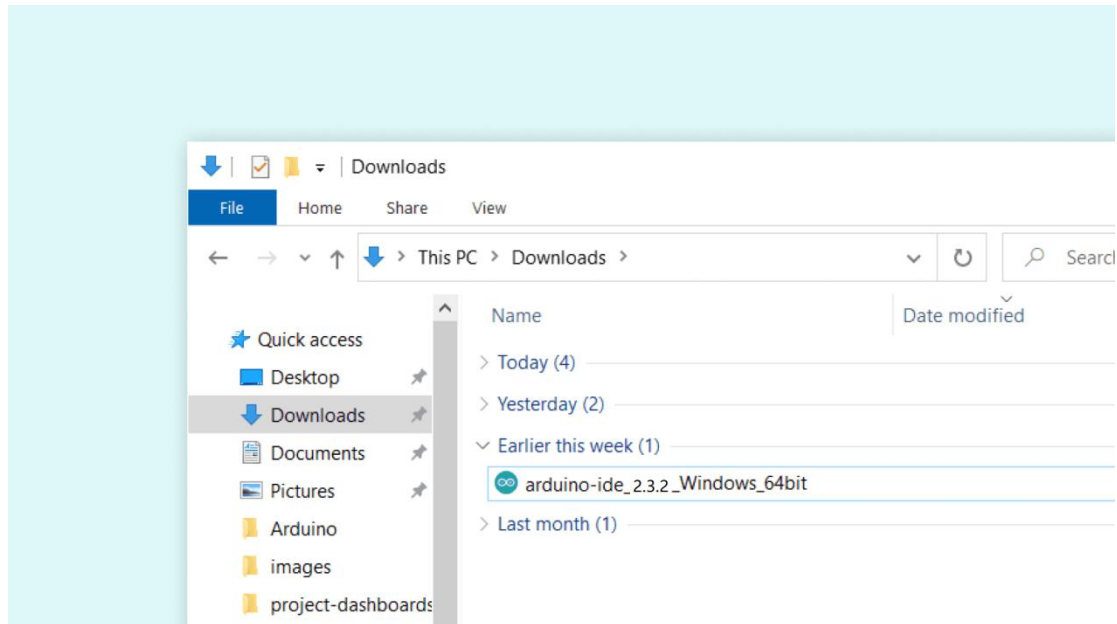
The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

DOWNLOAD OPTIONS

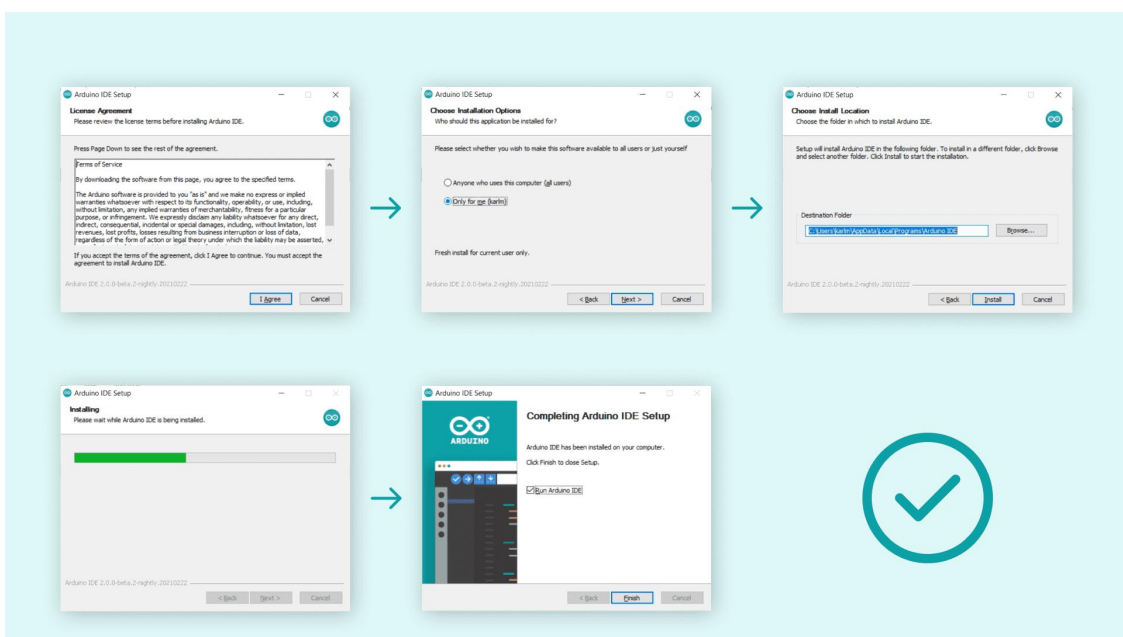
- Windows** Win 10 and newer, 64 bits
- Windows** MSI installer
- Windows** ZIP file
- Linux** AppImage 64 bits (X86-64)
- Linux** ZIP file 64 bits (X86-64)
- macOS** Intel, 10.15: "Catalina" or newer, 64 bits
- macOS** Apple Silicon, 11: "Big Sur" or newer, 64 bits

[Release Notes](#)

Windows コンピュータに Arduino IDE 2.x.x をインストールするには、ソフトウェアページからダウンロードしたファイルを実行するだけです。



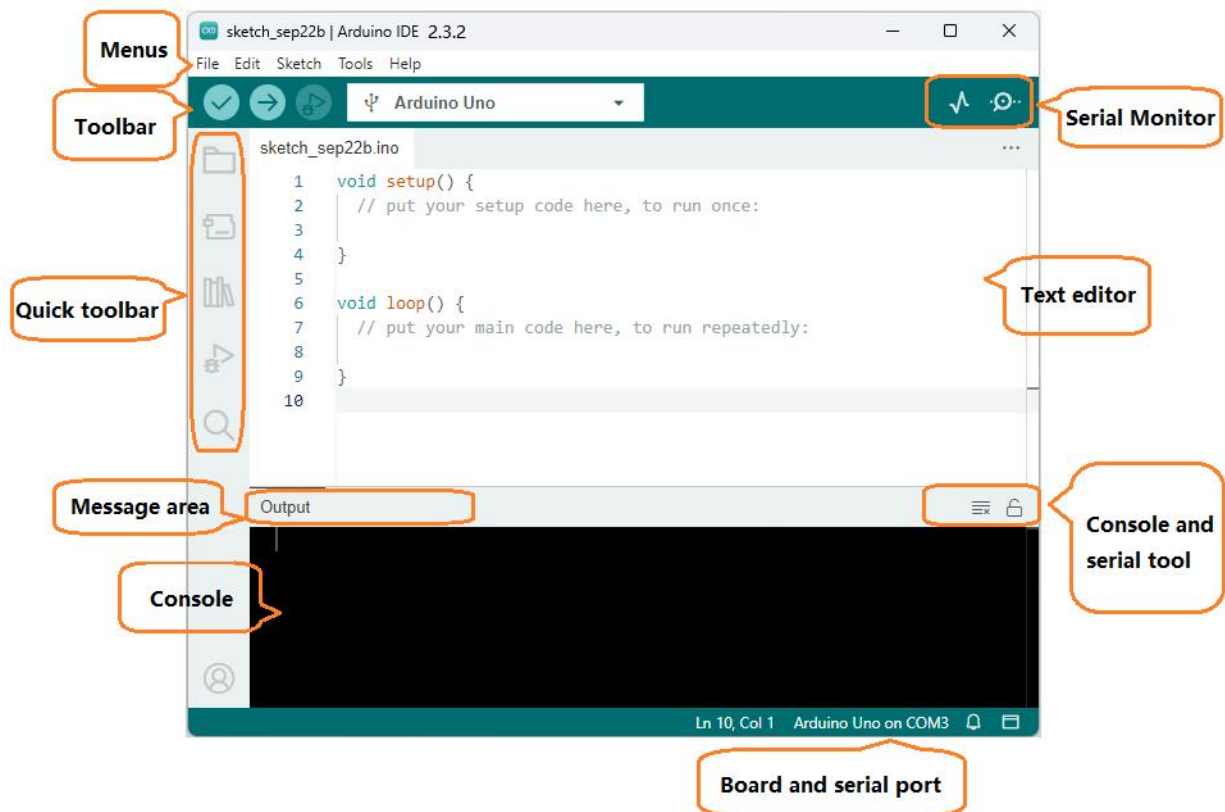
ポップアップウィンドウの指示に従ってインストールプロセスを完了してください。



スタートアップアイコン：

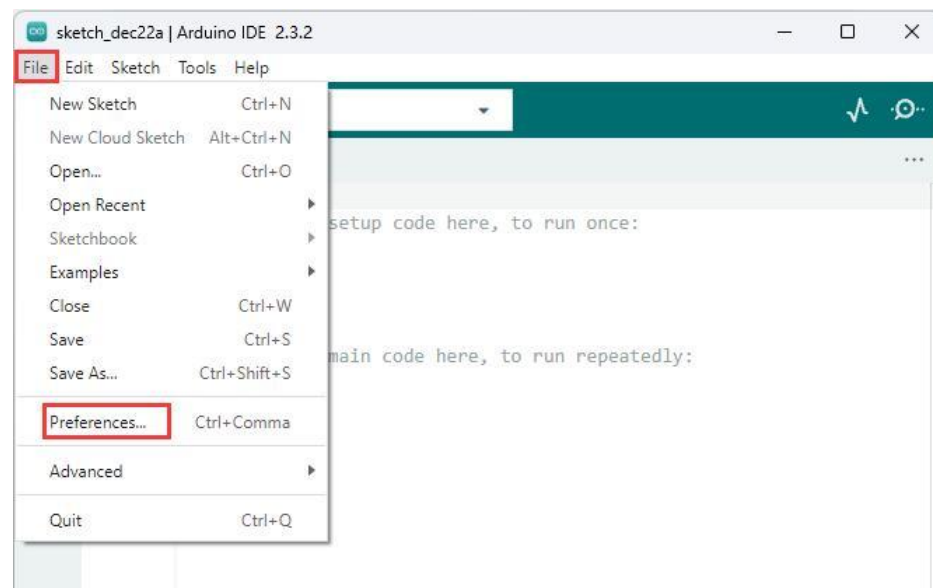


Arduino IDE のホームページ



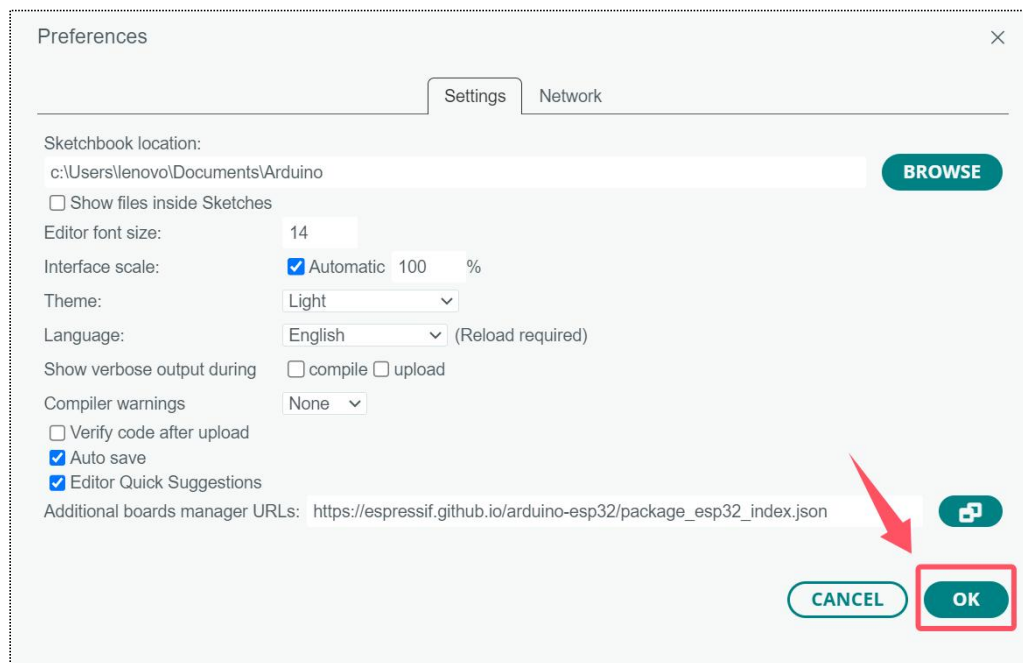
1.2 Arduino IDE 用の ESP32 開発環境をインストールする

Arduino IDE を開き、“File” をクリック> “Preferences”，以下のように表示されます。

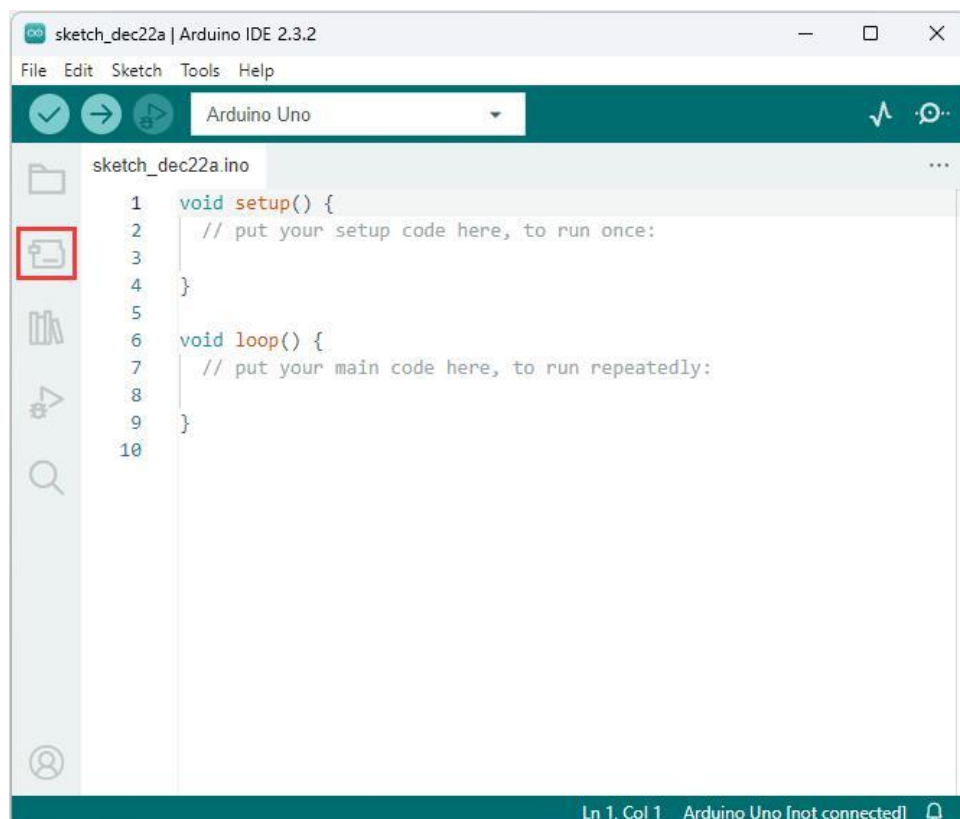


ポップアップウィンドウを下にスクロールし、「**Additional boards manager URLs**」テキストボックスに以下のリンクをコピーします。「**OK**」をクリックします。

https://espressif.github.io/arduino-esp32/package_esp32_index.json



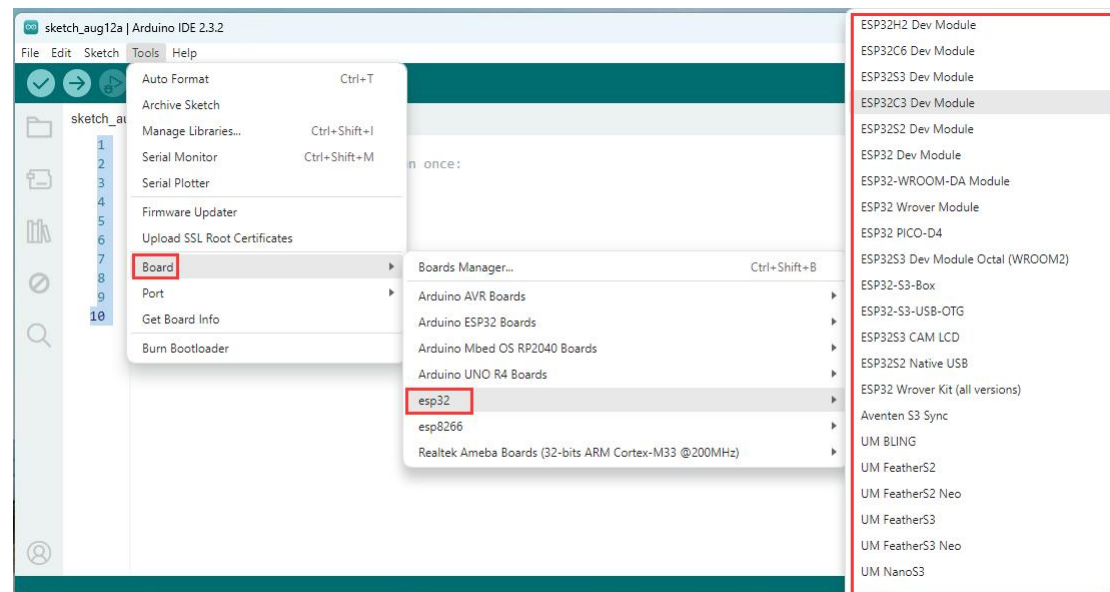
“**Boards Manager**”をクリックします:



ボードマネージャーで ESP32 を検索し、3.0.4 バージョンの esp32 by Espressif Systems を選択してインストールをクリックします。



ESP32 ボードを正常にインストールしたら、"Tools"->"Board"->"ESP32" メニューをクリックし、**ESP32C3 DVE Module** を選択します。



1.3 ライブラリファイルをインストールする

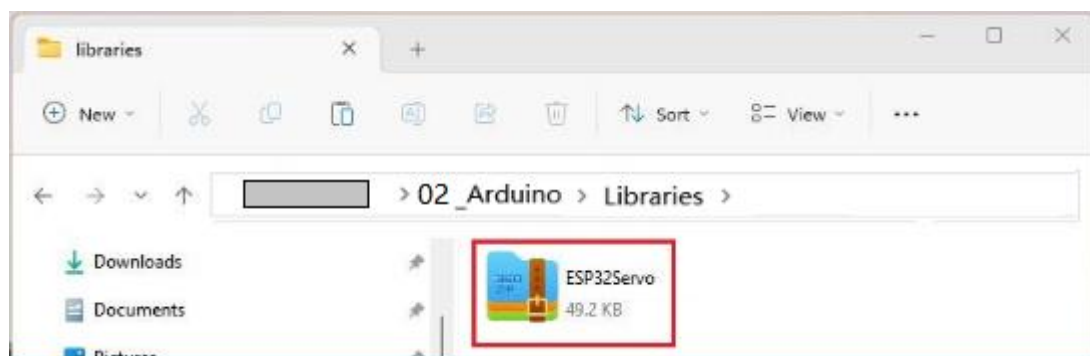
方法 1:

Arduino IDE のライブラリマネージャーを開き、検索バーに「ESP32Servo」と入力し、ESP32Servo とバージョン 3.0.5 と記載されたライブラリファイルを選択してインストールをクリックします。

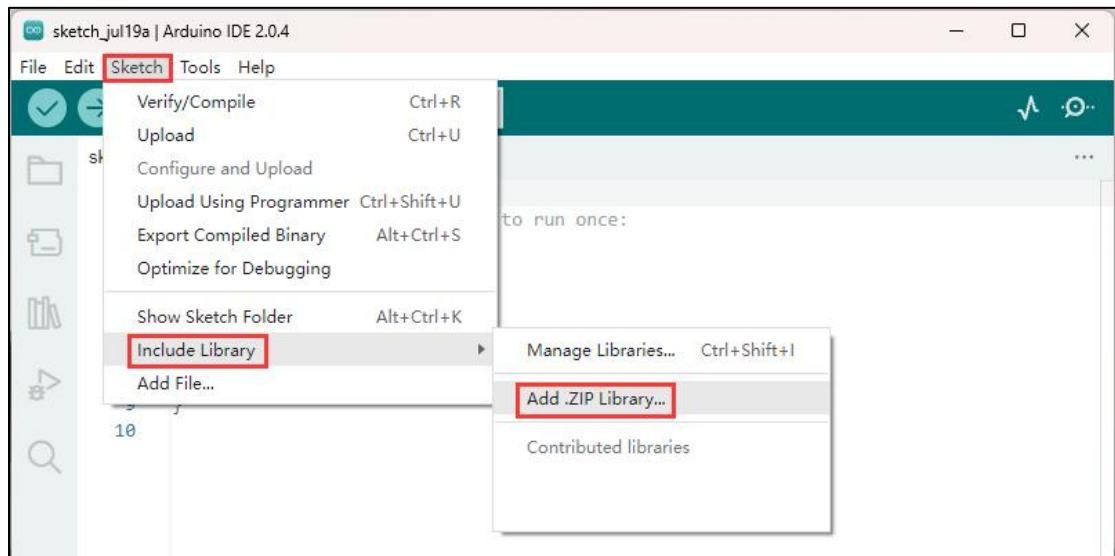


方法 2:

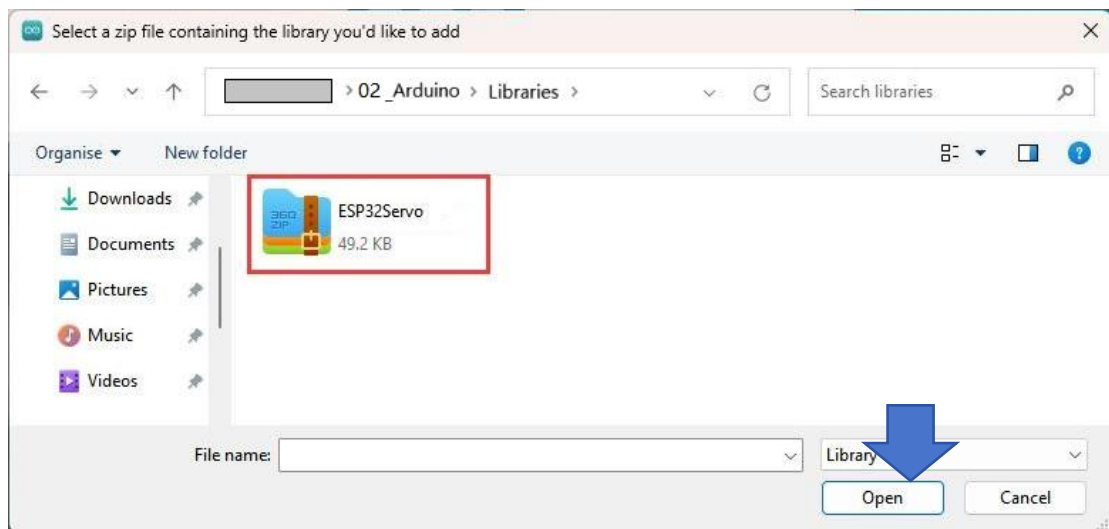
チュートリアルパッケージ内の次のフォルダにライブラリ zip ファイルを用意しています：



Arduino IDE からライブラリ zip ファイルをインポートする手順:

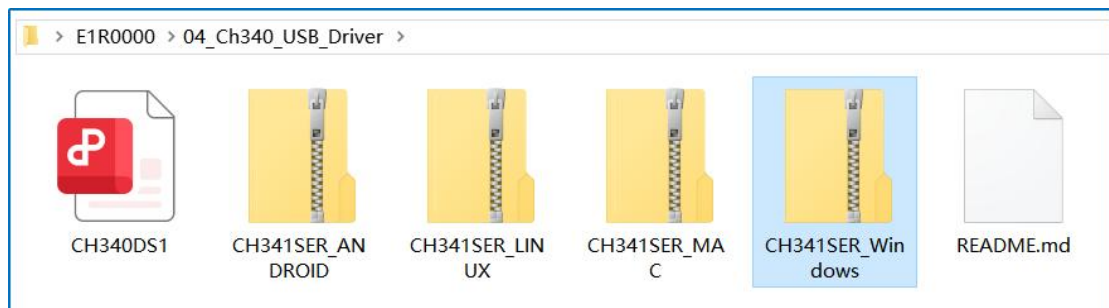


「Open」をクリックしてライブラリファイルをインストールします。



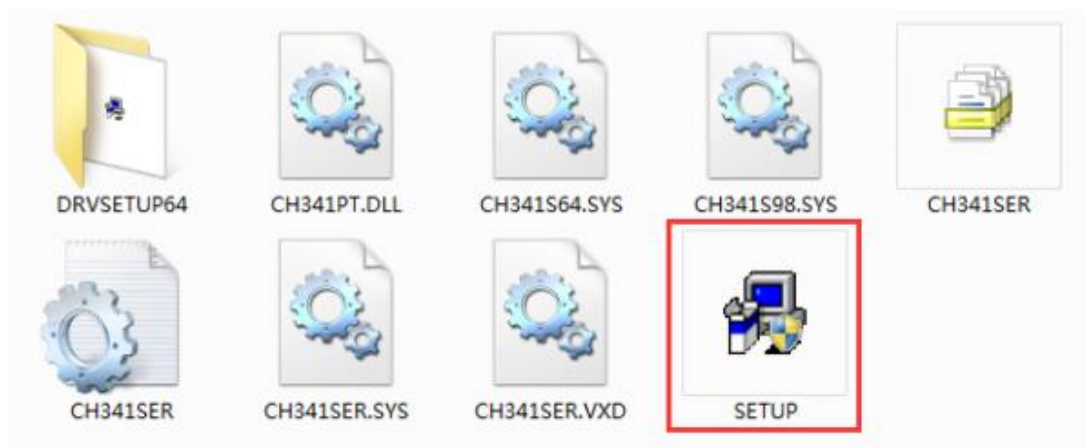
1.4 CH340 USB ドライバをインストールする

ドライバー ファイルはチュートリアル パッケージで提供されています:

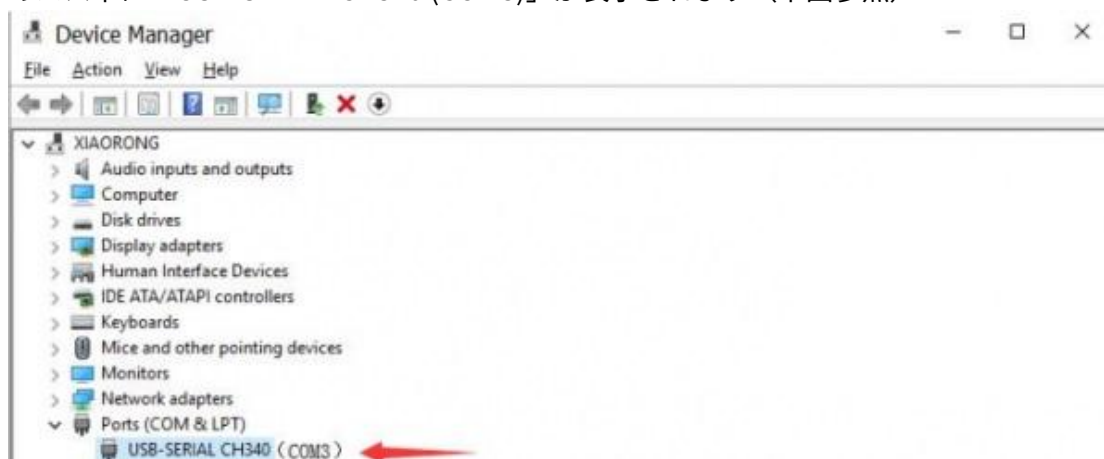


Windows システムにドライバーをインストールする

提供されたファイルを解凍し、「SETUP」をダブルクリックしてインストーラーを実行します:



ドライバーが正常にインストールされている場合、ESP32 ボードを USB ケーブルでコンピュータに接続すると、「コンピュータ」→「プロパティ」→「デバイスマネージャー」のパス下に「USB-SERIAL CH340 (COM3)」が表示されます（下図参照）:



注: ドライバー表示記号は「USB-SERIAL CH340 (COMx)」です。"x"は任意の数字であり、

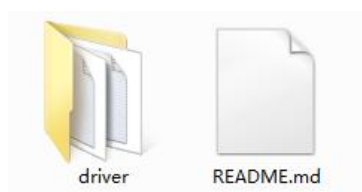
コンピュータが ESP32 デバイスに割り当てた番号によって異なります:

Linux システムにドライバーをインストールする

ドライバーはほぼ確実に Linux カーネルに組み込まれており、接続するだけで動作する可能性が高いです。動作しない場合は、Linux 用 CH340 ドライバーをダウンロードできます（ただし、「組み込み」のドライバーを使用できるように Linux インストールをアップグレードすることをお勧めします）。

自分でインストールする必要がある場合は、zip パッケージ内の「README.md」ファイ

ルを確認してください。



MAC システムにドライバーをインストールする

ドライバーのダウンロードリンク：

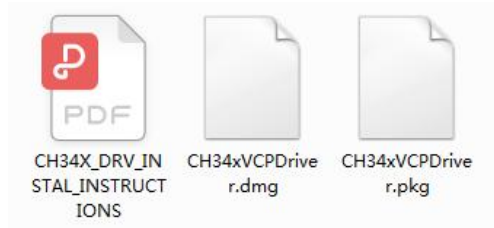
http://www.wch-ic.com/downloads/CH341SER_EXE.html



relation files

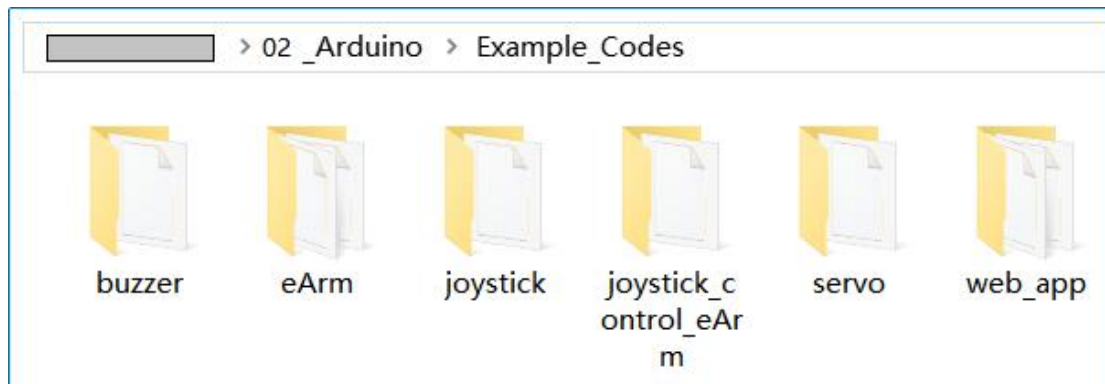
file name	file content
CH341SER.ZIP	CH340/CH341 USB to serial port Windows driver, supports Windows XP/Vista/7/8/8.1/10/11/ SERVER 2003/2008/2012/2016/2019/2022 -32/64bit, Microsoft WHQL Certified, supports USB to 3-line and 9-line serial port.
CH341SER_LINUX.ZIP	Linux driver for USB to serial port, supports CH340 and CH341, supports 32/64-bit operating systems.
CH341SER_MAC.ZIP	For CH340/CH341/CH342/CH343/CH344/CH347/CH9101/CH9102/CH9103/CH9104/CH9143, USB to serial port VCP vendor driver of macOS , supports OS X 10.9~10.15 , OS X 11 (Big Sur) and above , contains installation guide documents.
CH340DS1.PDF	CH340 datasheet, USB bus converter chip which converts USB to serial port, printer port, IrDA SIR etc. Integrated clock, supports various operating systems, chip information can be customized, this datasheet is about USB to serial port and USB to Infrared Adapter SIR.
CH341DS1.PDF	CH341 datasheet, USB bus converter chip with multiple communication interfaces, such as USB to serial port/parallel port/printer port/I2C interface etc. Drivers support for Windows/Linux/Android/Mac, etc. The datasheet is the description of USB to serial port and printer port.
CH341SER_ANDROID.ID.ZIP	CH340/CH341/CH342/CH343/CH344/CH347/CH9101/CH9102/CH9103/CH9104/CH9143 USB to serial port Android driver-free application library and software, for Android OS 4.4 and above in USB Host mode, without loading Android kernel driver and without root access operation. Includes apk installer, lib library file (Java Driver), App Demo routines (USB to UART Demo project SDK).

ウェブサイトからドライバーをダウンロードし、ファイルをローカルのインストールディレクトリに解凍してください。PDF インストールガイドと 2 種類の形式のインストールパッケージが入手できます。詳細は PDF インストールガイドをご参照ください。



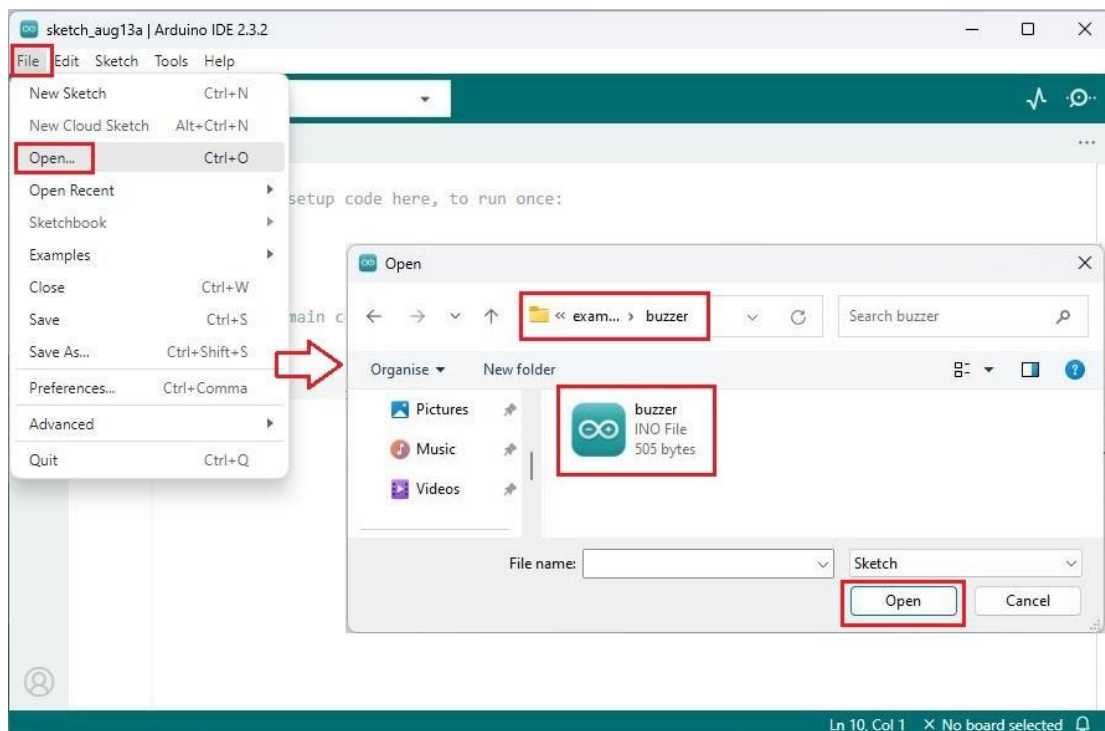
1.5 コード例

サンプル コードは、チュートリアル パッケージ内の次のフォルダーに保存されます:

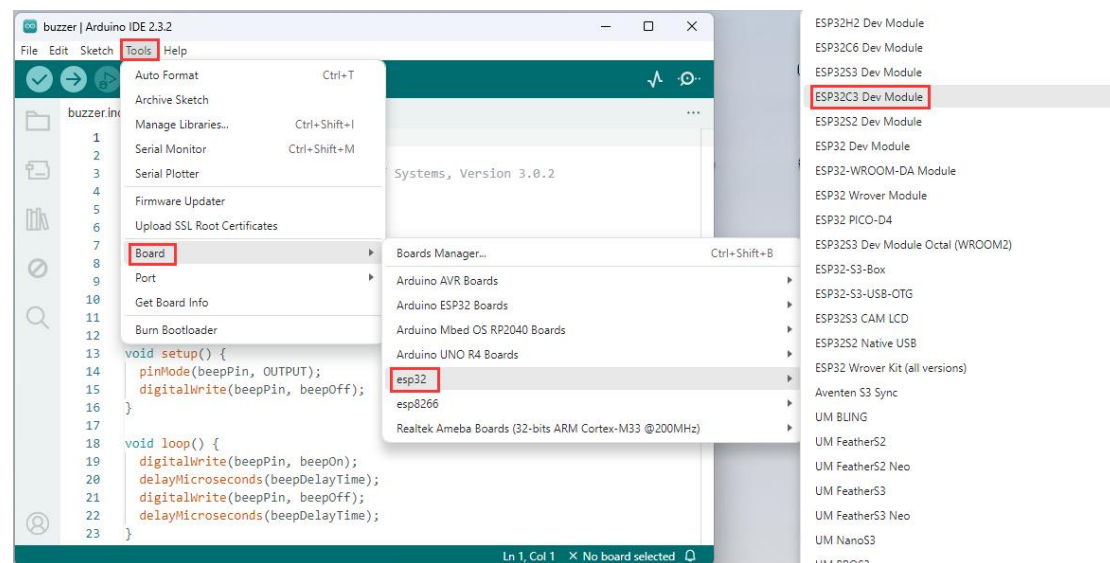


1.5.1 オンボードブザー

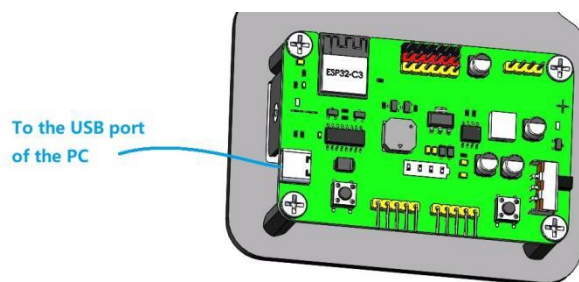
Arduino IDE で「buzzer」のサンプルコードを開きます:



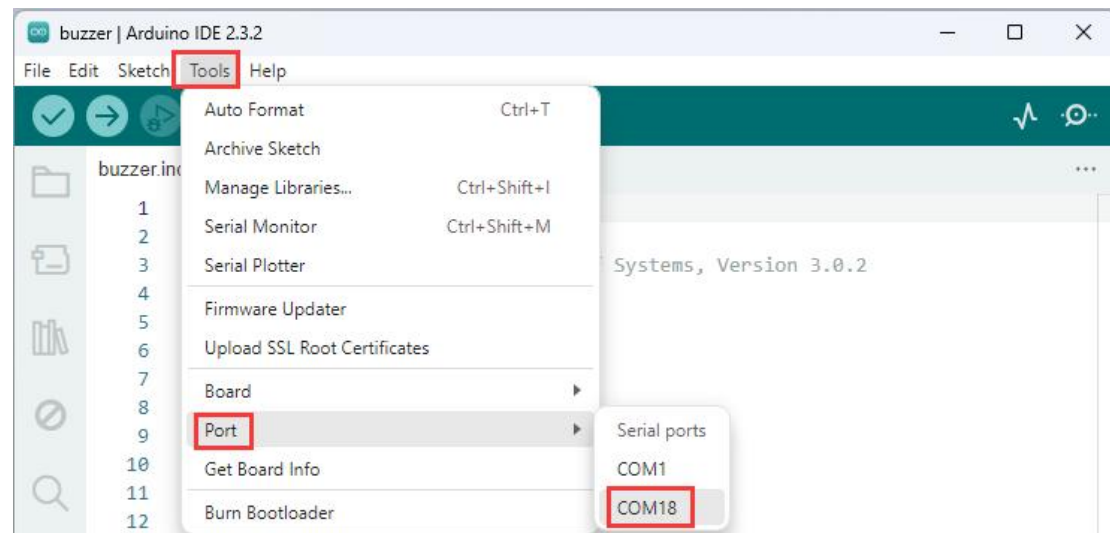
ボードの種類を選択（ESP32C3 Dev Module）：



以下に示すように、USB ケーブルを使用して PC と eArm を接続します：

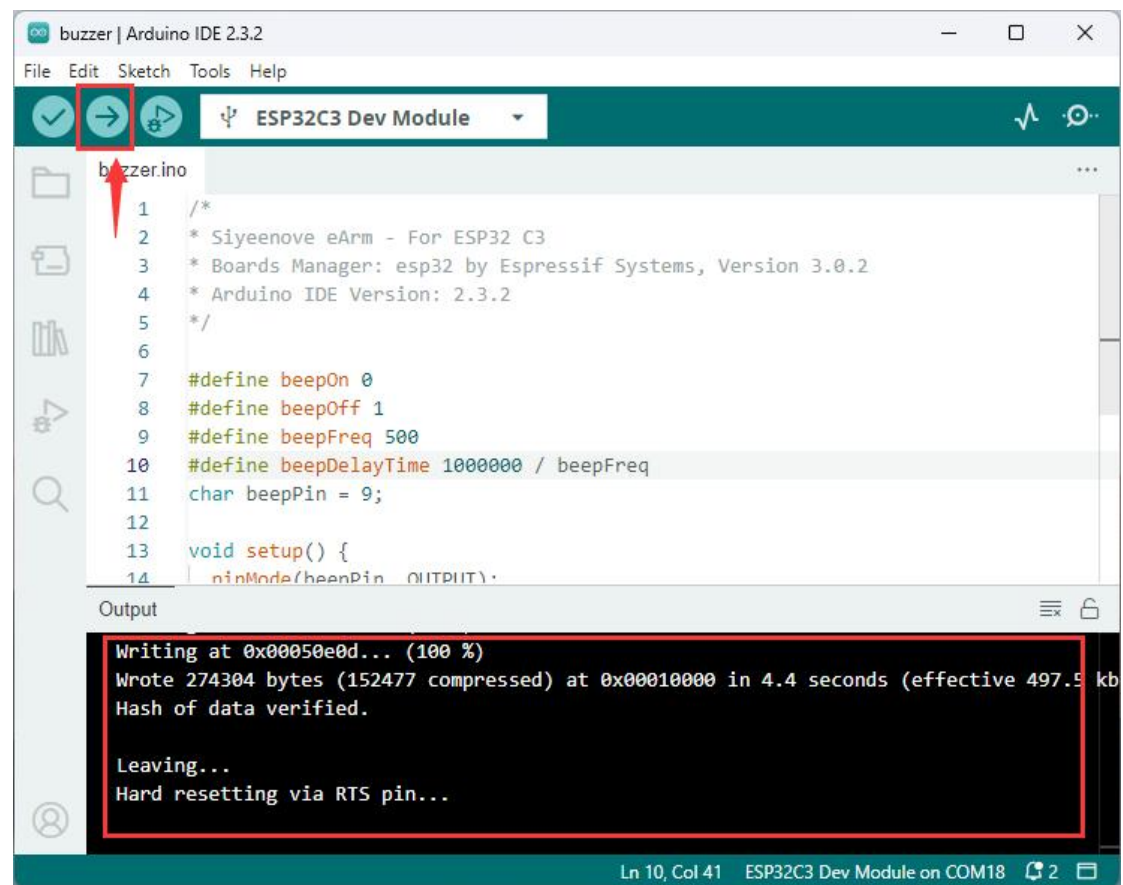


ポートを選択：

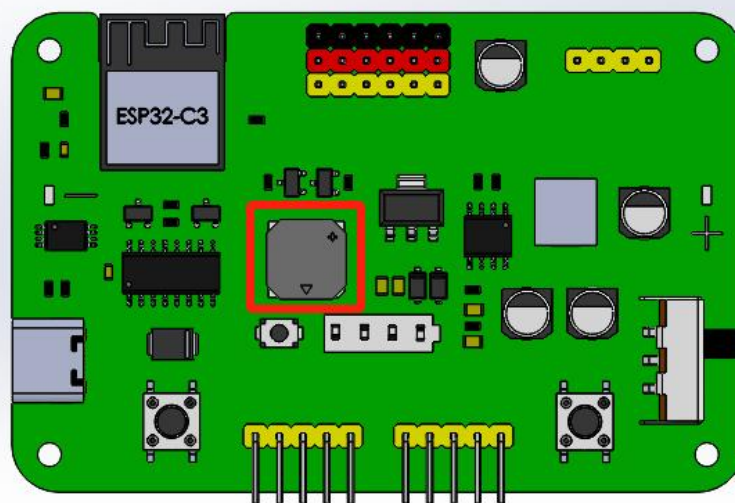


注：上の写真のポートは COM18 です。お使いのコンピュータによっては異なる番号のポートが使用されている場合があります。コンピュータのデバイスマネージャーで割り当てられたポート番号に従ってポートを選択してください。

クリックしてコードをアップロード：



コードが正常にアップロードされると、ESP32 ボード上のブザーが鳴り続けます：

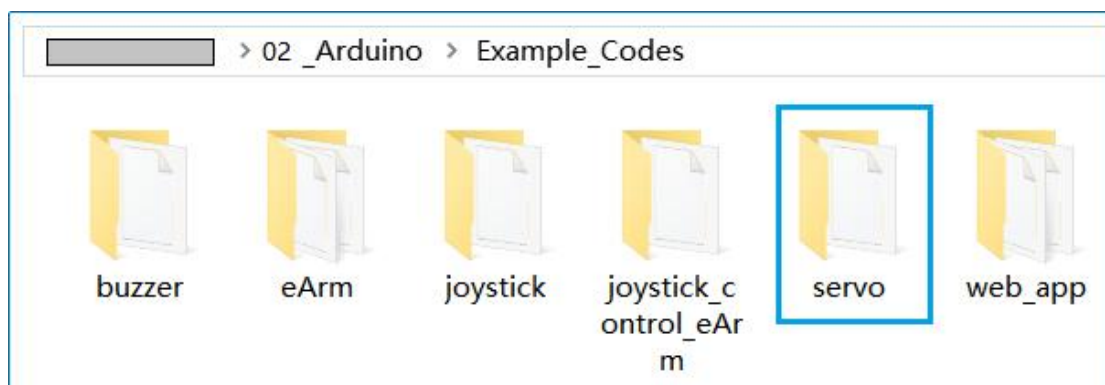


コードの説明：

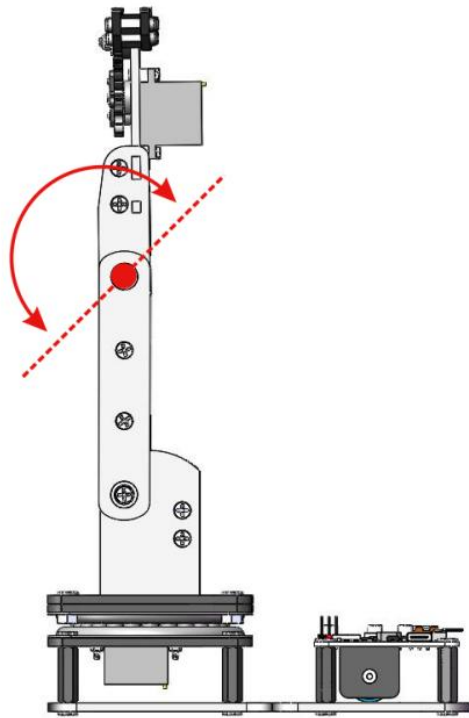
<code>#define beepOn 0</code>	定数「beepOn」を値 0 で定義する
<code>#define beepDelayTime 1000000 / beepFreq</code>	定数「beepDelayTime」を値「1000000 / beepFreq」で定義する
<code>char beepPin = 9;</code>	文字変数「beepPin」を値 9（ブザーの制御ピン にマッピング）で定義する
<code>void setup() { ... }</code>	Arduino IDE の組み込み関数、電源投入時に 1 度だけ実行される
<code>pinMode(beepPin, OUTPUT);</code>	ピン 9 を出力モードに設定する
<code>digitalWrite(beepPin, beepOff);</code>	ピン 9 にハイレベルを出力しブザーをオフにする
<code>void loop() { ... }</code>	Arduino IDE の組み込み関数、電源投入中は常に ループ実行される
<code>delayMicroseconds(beepDelayTime);</code>	Arduino IDE の組み込み関数、マイクロ秒単位の 遅延処理

1.5.2 サーボを駆動する

「servo」コードを ESP32 制御ボードにアップロードするには、セクション 1.5.1 を参照してください:



コードが正常にアップロードされると、eArm のサーボ C は 1 サイクルで 0 ～ 180、180 ～ 0 に振動します：

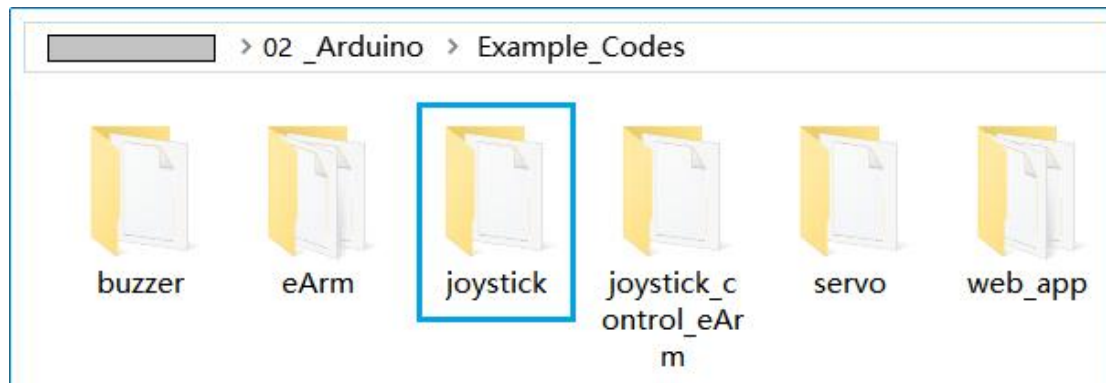


コードの説明：

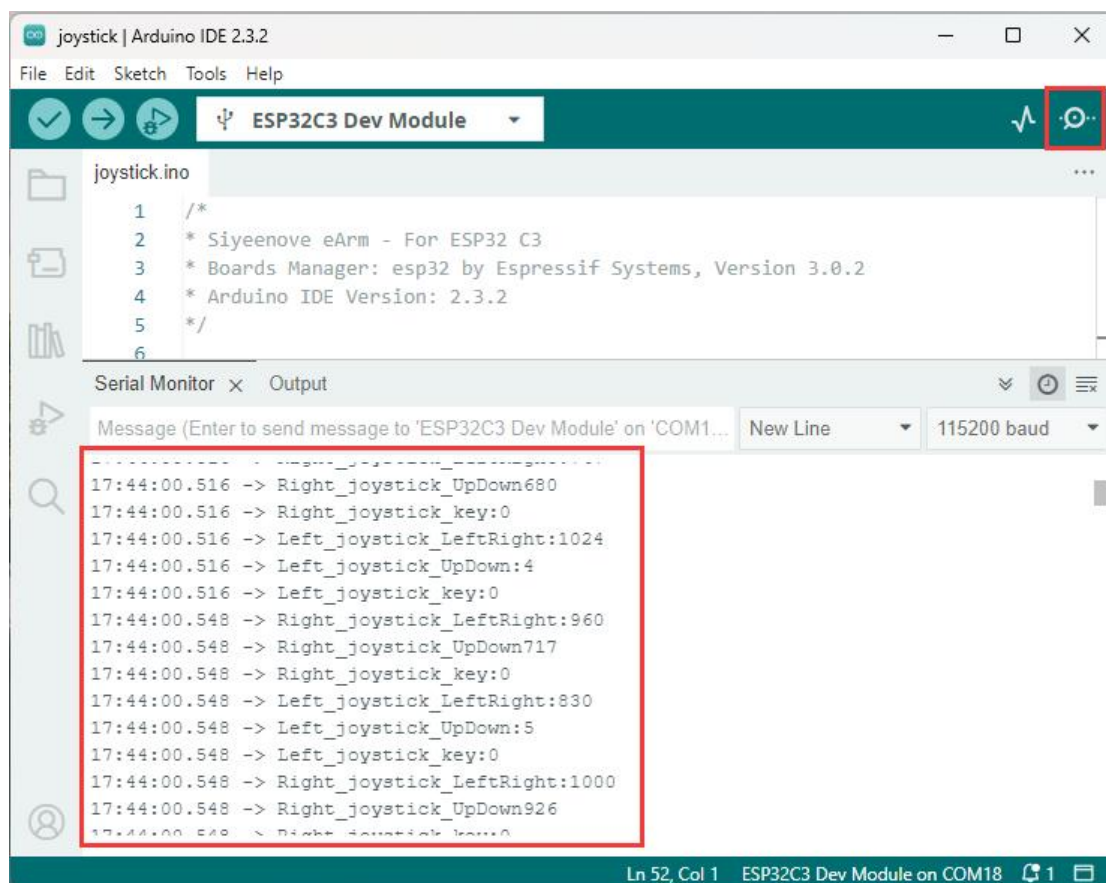
<code>#include <ESP32Servo.h></code>	ESP32 サーボドライブライブラリをインクルードする
<code>Servo A servo;</code>	サーボクラスを定義し Servo1 を駆動する
<code>ESP32PWM::allocateTimer(0);</code>	サーボドライブライブラリにタイマー 0 を使用させる
<code>A servo.setPeriodHertz(50);</code>	駆動サーボ A servo のパルス駆動周波数を 50Hz に設定する
<code>A servo.attach(A servoPin, 500, 2400);</code>	駆動サーボ A servo に A servoPin ピンを使用させ、最新パルス幅 500 マイクロ秒、最大パルス幅 2400 マイクロ秒で駆動する
<code>A servo.write(a servoAngle);</code>	駆動サーボ A servo を角度 a servoAngle に回転させる
<code>for(statement1;condition;statement2){ ... }</code>	for ループ文：statement1 を実行する度に条件が真か判定し、真なら中括弧内の文を実行、偽ならループを中断。次に statement2 を実行後、再度条件判定を行いループを繰り返す。

1.5.3 ジョイスティックの値を読み取る

「ジョイスティック」コードを ESP32 コントロール ボードにアップロードするには、セクション 1.5.1 を参照してください:



次に、Arduino IDE のシリアル ポート モニターを開き、ハンドルを左、右、上、下に振るか、ジョイスティックを押し下げると、シリアル ポートにジョイスティックの値が出力されます:

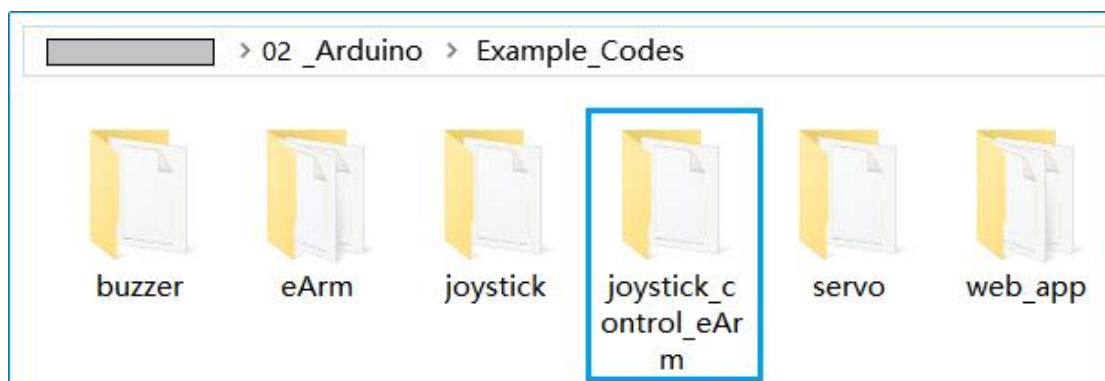


コードの説明：

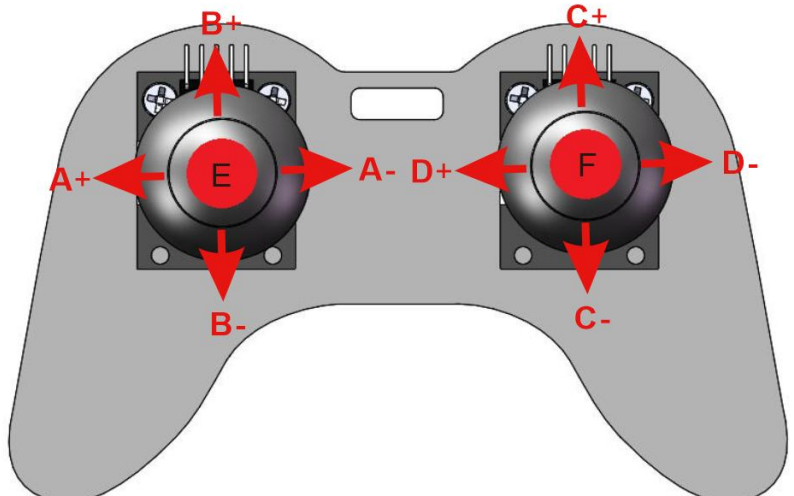
<code>Serial.begin(115200);</code>	シリアルポートのボーレートを 115200 に設定
<code>pinMode(left_joystick_KeyPin, INPUT);</code>	左ジョイスティックのキーピンを「left_joystick_KeyPin」に設定し、モードを出力に設定
<code>pinMode(right_joystick_KeyPin, INPUT_PULLUP);</code>	右ジョイスティックのキーピンを「right_joystick_KeyPin」に設定し、モードを出力に、プルアップ抵抗付きで設定
<code>Serial.print("Left_joystick_LeftRight:");</code>	シリアルポートで改行なしの文字列を出力
<code>Serial.println(analogRead(left_joystick_LeftRightPin));</code>	シリアルポートでアナログ値を出力し改行
<code>analogRead(left_joystick_LeftRightPin);</code>	左ジョイスティックの左右揺れの値を読み取る
<code>delay(1000);</code>	1000 ミリ秒の遅延

1.5.4 ジョイスティックで eArm を制御する(記録機能付き)

1.5.1 節に従って、「joystick_control_eArm」コードを ESP32 制御ボードにアップロードしてください。



ジョイスティック制御ロボットアームのデモンストレーション：

	
A+: Aservo を制御し、ロボットアームを左に回転させます。	A-: Aservo を制御し、ロボットアームを右に回転させます。
B+: Bservo を制御し、ロボットアームの大アームを前へ振り出します。	B-: Bservo を制御し、ロボットアームの大アームを後ろへ振り出します。
C+: Cservo を制御し、ロボットアームの小アームを前へ振り出します。	C-: Cservo を制御し、ロボットアームの小アームを後ろへ振り出します。
D+: Dservo を制御し、ロボットアームのグリッパーを開きます（緩めます）。	D-: Dservo を制御し、ロボットアームのグリッパーを閉じます（締めます）。
E：記録機能 - E ボタンを 1 回押すと、1 つの動作を記録します（最大 20 個の動作を順番に記録可能）。押すたびにブザーが短く 1 回鳴ります。記録数が最大に達すると、ブザーが長く 1 回鳴ります。	F：再生 — 動作シーケンスの記録が完了したら、F ボタンを 1 回押すと、記録された動作を連続で繰り返し再生（ループ再生）します。ブザーが 1 回鳴り、機械臂が記録された動作をループで繰り返します。 ループを終了するには、F ボタンを長押ししてください。ブザーが長く 1 回鳴り、再生が停止したことをお知らせします。

注 動的メモリを使用しているため、電源オフ後は記録されたすべての動作が失われます！
 コードから、デフォルトの最大記録動作数を変更することもできます。

```

53  const int act_max=10000; //Default 100 action,4 the Angle of servo
54  int act[act_max][4]; //Only can change the number of action
55  int num=0,num_do=0;
  
```

Output

```

cmd /c IF 0==1 COPY /y "C:\\Users\\11235\\AppData\\Local\\Arduino15\\packages\\esp32\\tools\\openocd
cmd /c IF 0==1 COPY /y "C:\\Users\\11235\\AppData\\Local\\Arduino15\\packages\\esp32\\hardware\\esp3
cmd /c IF 0==1 COPY /y "C:\\Users\\11235\\AppData\\Local\\Arduino15\\packages\\esp32\\hardware\\esp3

Using library ESP32Servo at version 3.0.5 in folder: C:\\Users\\11235\\Documents\\Arduino\\libraries\\ESP3
"C:\\Users\\11235\\AppData\\Local\\Arduino15\\packages\\esp32\\tools\\riscv32-esp-elf-gcc\\esp-2021r
Sketch uses 251604 bytes (19%) of program storage space. Maximum is 1310720 bytes.
Global variables use 174428 bytes (53%) of dynamic memory, leaving 153252 bytes for local variables.
  
```

Arduino IDE でコードをコンパイルした後、出力ログを確認してください。記録動作数を 10,000 に変更しても、動的メモリの 53%しか使用していません。つまり、さらに多くの動作を記録できるということです！

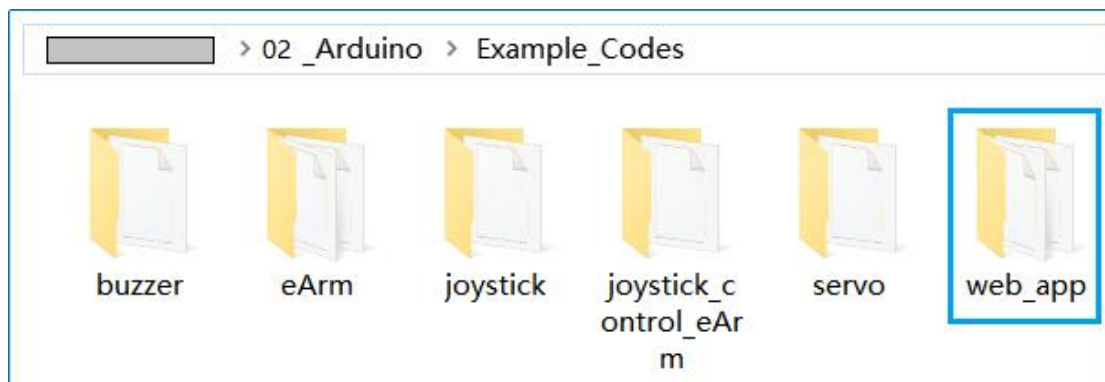
代碼解説：

<code>eArm arm;</code>	ロボットアームクラスを定義します。
<code>arm.ServoAttach(4,5,6,7);</code>	ロボットアームのサーボ A、B、C、D は、それぞれ制御ボードのピン 4、5、6、7 に接続されています。
<code>arm.JoyStickAttach(0,1,10,2,3,8);</code>	左ジョイスティックの X、Y、Z ピンは、それぞれ制御ボードのピン 0、1、10 に接続されています。右ジョイスティックの X、Y、Z ピンは、それぞれ制御ボードのピン 2、3、8 に接続されています。
<code>int act[x][y];</code>	2 次元整数配列を定義します。x は行パラメータ、y は列パラメータです。
<code>arm.down(speed);</code>	ロボットアームが下方向に振れます。speed は振れの速度です。
<code>arm.up(speed);</code>	ロボットアームが上方向に振れます。speed は振れの速度です。
<code>arm.left(speed);</code>	ロボットアームが左に回転します。speed は回転速度です。
<code>arm.right(speed);</code>	ロボットアームが右に回転します。speed は回転速度です。
<code>arm.clawClose(speed);</code>	ロボットアームのグリッパーが閉じます。speed は動作速度です。
<code>arm.clawOpen(speed);</code>	ロボットアームのグリッパーが開きます。speed は動作速度です。
<code>void turnUD(void);</code>	ジョイスティックでロボットアームの上下動作を制御します。
<code>void turnLR(void);</code>	ジョイスティックでロボットアームの左右回転を制御します。
<code>void claw(void);</code>	ジョイスティックでロボットアームのグリッパー開閉を制御します。
<code>void date_processing(int *x,int *y);</code>	ジョイスティックの X 軸および Y 軸データを処理し、各軸の最大振れデータを読み取ります。

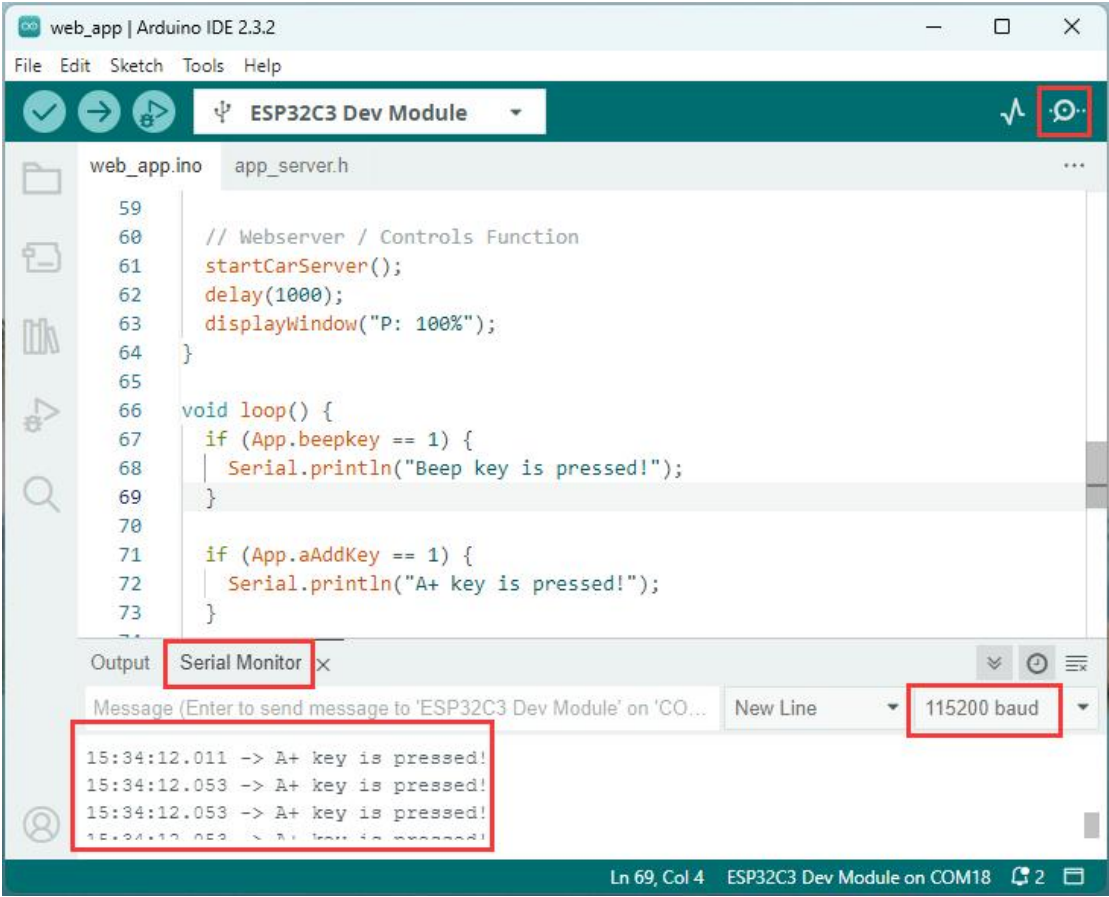
<code>void buzzer(int H,int L);</code>	ジョイスティック Z 軸ボタンのブザー機能。H はブザーがオンになる時間（マイクロ秒）、L はブザーがオフになる時間（マイクロ秒）です。
<code>arm.JoyStickL.read_z();</code>	左ジョイスティック Z 軸のbool値を読み取ります。
<code>arm.JoyStickR.read_z();</code>	右ジョイスティック Z 軸のbool値を読み取ります。
<code>void Re_action(void);</code>	ロボットアームの現在の動作を記録し、2次元配列に保存します。
<code>void Ex_action(void);</code>	2次元配列に記録されたすべての動作を実行します。
<code>void executionAction(int *angle,int speed);</code>	単一の動作を実行します。*angle はポインタ、speed は実行速度です。
<code>arm.JoyStickL.read_x();</code>	左ジョイスティック X 軸のデータを読み取ります。戻り値: 0～4059。
<code>arm.JoyStickL.read_y();</code>	左ジョイスティック Y 軸のデータを読み取ります。戻り値: 0～4059。
<code>arm.JoyStickR.read_x();</code>	右ジョイスティック X 軸のデータを読み取ります。戻り値: 0～4059。
<code>arm.JoyStickR.read_y();</code>	右ジョイスティック Y 軸のデータを読み取ります。戻り値: 0～4059。

1.5.5 Web アプリの値を読み取る

セクション 1.5.1 を参照して、「web_app」コードを esp32 コントロール ボードにアップロードしてください:



「[組立・取扱説明書](#)」を参照し、WiFi に「eArm」を接続して Web アプリを使用してください。その後、Arduino IDE のシリアルモニターを開き、Web アプリのボタンをクリックすると、シリアルモニターにボタンに対応する文字列が表示されます:



コードの説明：

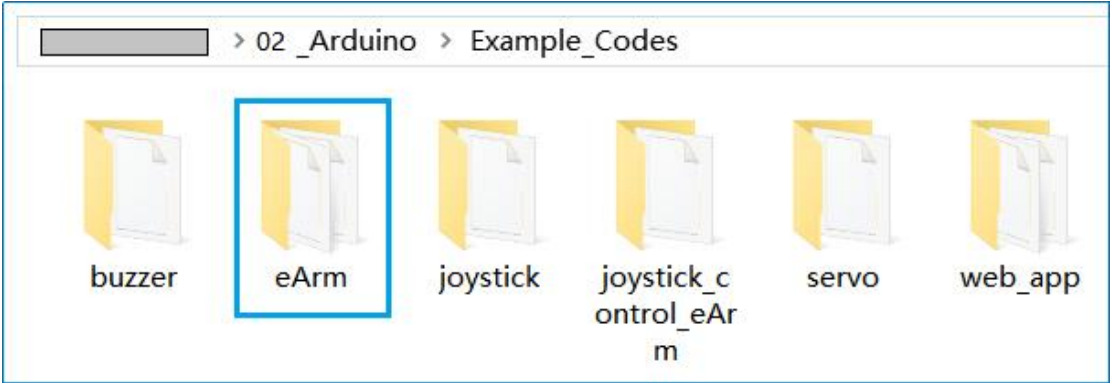
<code>#include "app_server.h"</code>	同一フォルダ内の「app_server.h」ヘッダファイルをインクルード
<code>bool ap = 1;</code>	ブール変数「ap」を定義し値 1 を代入
<code>const char* ssid = "eArm";</code>	定数文字ポインタを定義し値を代入
<code>int channel = 11;</code>	整数変数「channel」を定義し値 11 を代入
<code>WiFi.mode(WIFI_STA);</code>	Wi-Fi を「ステーション」モードに設定

<code>WiFi.begin(ssid, password);</code>	Wi-Fi 名とパスワードを設定し Wi-Fi を起動
<code>while(condition) {...}</code>	while 条件ループ文：条件が真の場合、中括弧内の文を実行；偽の場合ループを中断
<code>WiFi.status();</code>	Wi-Fi 接続状態を読み取る
<code>WiFi.localIP();</code>	「ステーション」モードで Wi-Fi IP アドレスを読み取る
<code>WiFi.mode(WIFI_AP);</code>	Wi-Fi を AP モードに設定
<code>WiFi.softAP(ssid, password, channel, hidden, maxconnection);</code>	AP モードで Wi-Fi 名、パスワード、チャンネル、名前の非表示設定、最大接続数を設定
<code>WiFi.softAPIP();</code>	AP モードの Wi-Fi IP アドレスを読み取る
<code>startCarServer();</code>	Web サーバーを起動
<code>displayWindow("P: 100%");</code>	ウェブページのフロントエンドに文字列を表示
<code>if(condition){ ... }</code>	if 条件文：条件が真の場合、中括弧内の文を実行；偽の場合実行しない
<code>App.beepkey</code>	ウェブページフロントエンドのブザーボタンの状態変数値

1.5.6 ロボットアームの使用

このサンプルコードは当社工場ファームウェアの完全なソースコードです。以下の方法で制御ボードにアップロードすると、工場出荷時の機能を完全に復元できます。

「1.5.1 章」を参照し、"eArm"コードを ESP32 制御ボードにアップロードしてください:



eArm コードのアップロード後、前述の「[組立・取扱説明書](#)」で説明されている通り、このロボットアームを制御できるようになります。

コードの説明：

<code>TaskHandle_t TASK_HandleOne = NULL;</code>	スレッドハンドルを作成
<code>xTaskCreate(...);</code>	スレッドを初期化
<code>void TASK_ONE(void* param) { ... }</code>	スレッド関数 (loop()関数と並列実行される)

2

ファームウェアの書き込み

ファームウェアファイル（.bin や.hex 形式）としてデバイスの「工場リセットプログラム」を直接ユーザーに提供し、Arduino 開発環境を構築せずにデバイスに書き込むことで復元プロセスを完了できます。

1. 従来の方法の問題点

通常、Arduino 経由でデバイスを復元するには、ユーザーは以下が必要でした：

- ▶ Arduino IDE または関連開発ツールのインストール

- ▶ ソースコード（.ino ファイル）のダウンロード（追加ライブラリの設定や依存関係解決が必要な場合あり）

- ▶ コンパイルとデバイスへのアップロード

このプロセスは非技術者にとってハードルが高く、環境関連の問題で失敗する可能性があります。

2. ファームウェア直接提供の利点

ユーザープロセスの簡素化：

- ▶ プリコンパイル済みファームウェアファイル（例：firmware.bin）を提供し、シリアルフラッシャーなどの簡単なツールで 1 ステップで書き込み可能

- ▶ Arduino IDE のインストール、コンパイルエラーの解決、ライブラリ競合の管理が不要

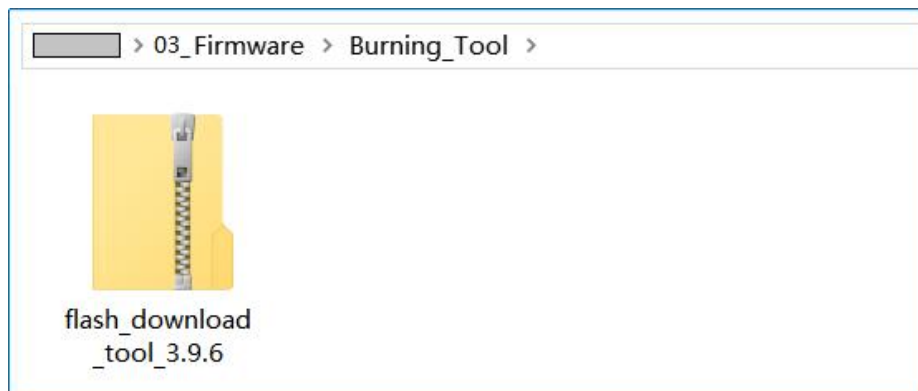
使用例：

- ▶ 不調なデバイスを工場出荷状態に復元

- ▶ 複数デバイスへの同一ファームウェアの一括書き込み（効率化）

2.1 書き込みツール

Burning Tool は次のフォルダに保存されています:



また、公式ウェブサイトから最新バージョンの Burning Tool をダウンロードすることもできます:

<https://www.espressif.com/en/support/download/other-tools>

Flash Download Tools

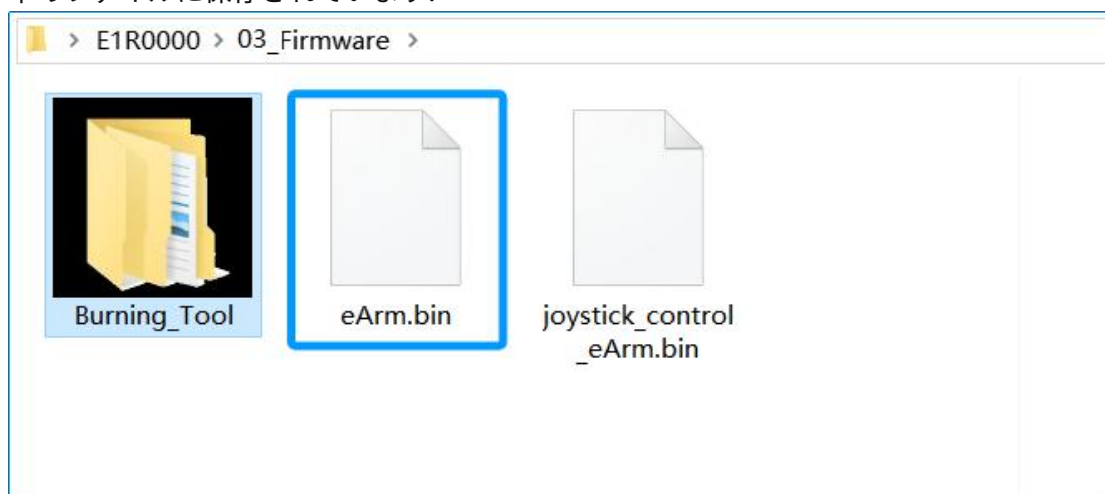
Expand all +

Please refer to Flash Download Tool User Guide to download this tool.

Title	Platform	Version	Release Date ▾	Download
+ Flash Download Tools	HTML	latest	2024.12.18	

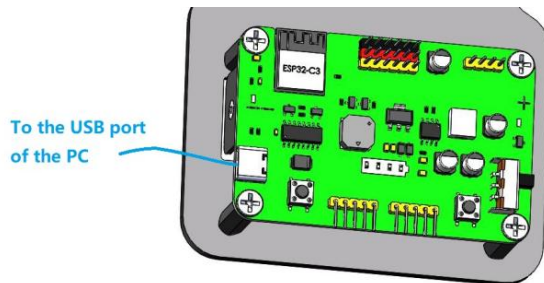
2.2 eArm ファームウェア（工場出荷時ファームウェア）

このファームウェアは、工場出荷時のロボットアームと全く同じ機能を備えています。工場出荷時の機能が失われた場合、このファームウェアを使用して機能を復元できます。以下のファイルに保存されています:

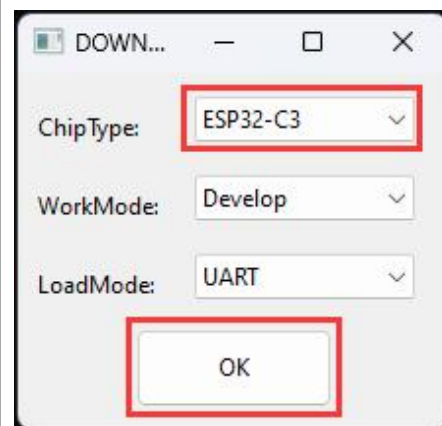


2.2.1 eArm ファームウェアの書き込み

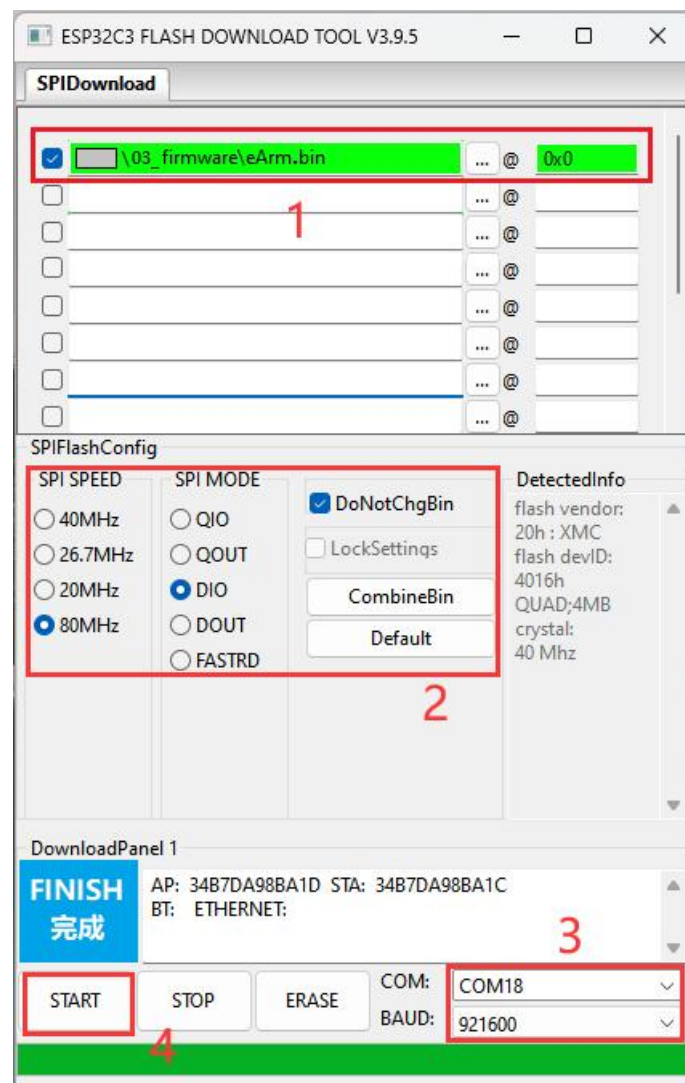
USBケーブルを使用してeArmをPCに接続します：



書き込みツールを起動します：



書き込み対象のファームウェアを選択し、書き込み先アドレスを正しく入力します。



CH340 ドライバーがインストールされており、バッテリーが正しく接続され、eArmの電源スイッチがオンになっている必要があります。そうでない場合、COMポートは検出されません。

1：書き込みオプションの「」にチェックを入れ、「」をクリックし、本チュートリアルで提供されている「**eArm.bin**」ファームウェアファイルを選択します。その後、「@」の後に書き込みアドレス（**0x0**）を正しく入力してください：

ファイル名	書き込みアドレス
eArm.bin	0x0

2：SPI SPEED で **80MHz**、SPI MODE で **DIO** を選択し、 **DotNotChgBin** にチェックを入れます。

3：制御ボード用のシリアルポートを選択します。ここでは COM18 です（お使いのコンピュータでこのデバイスに実際に割り当てられているポートに基づいて選択してください）。ボーレートを **921600** に設定します。書き込みに失敗した場合は、ボーレートを **115200** などに下げてください。

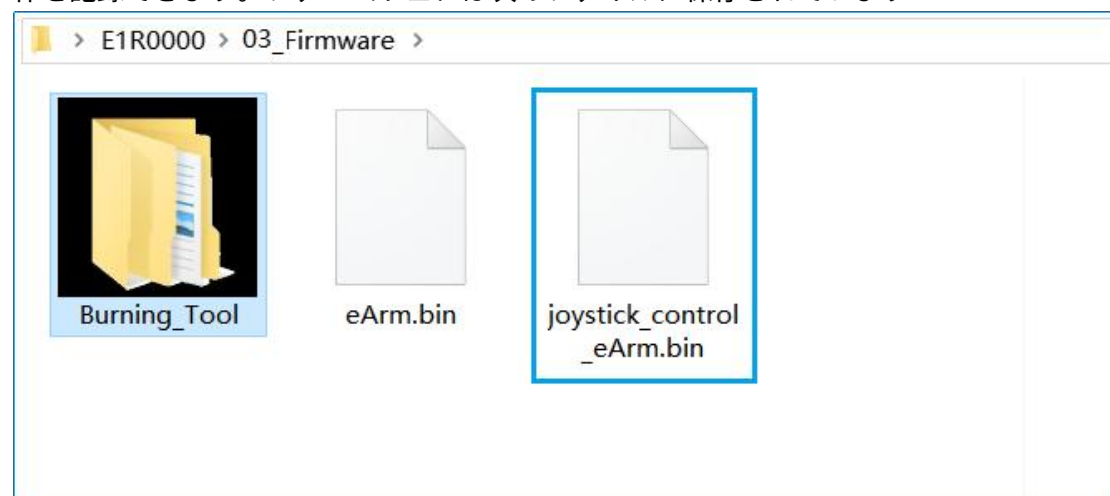
4：クリックしてファームウェアの書き込みを開始します。

書き込みが完了したら、[\[組立・取扱説明書\]](#)の手順に従ってロボットアームを操作できます！

注記：ファームウェアの書き込みが完了したら、RST（リセット）ボタンを押すか、USB ケーブルを抜いてから、再度電源を入れてください。

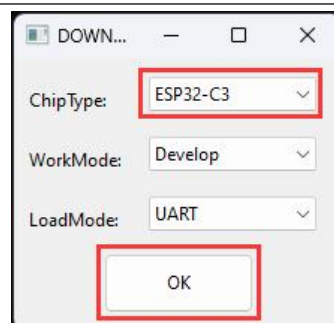
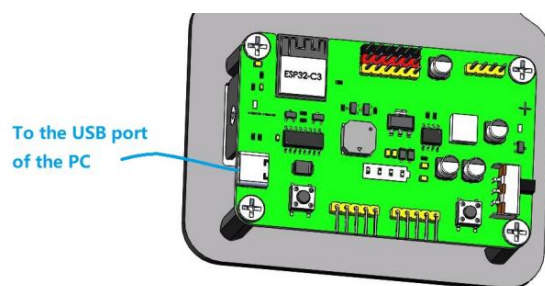
2.3 ジョイスティック制御ファームウェア（記録機能付き）

このファームウェアは、セクション 1.5.4 のコードと同じ機能を備えており、20 個の動作を記録できます。ファームウェアは次のファイルに保存されています：

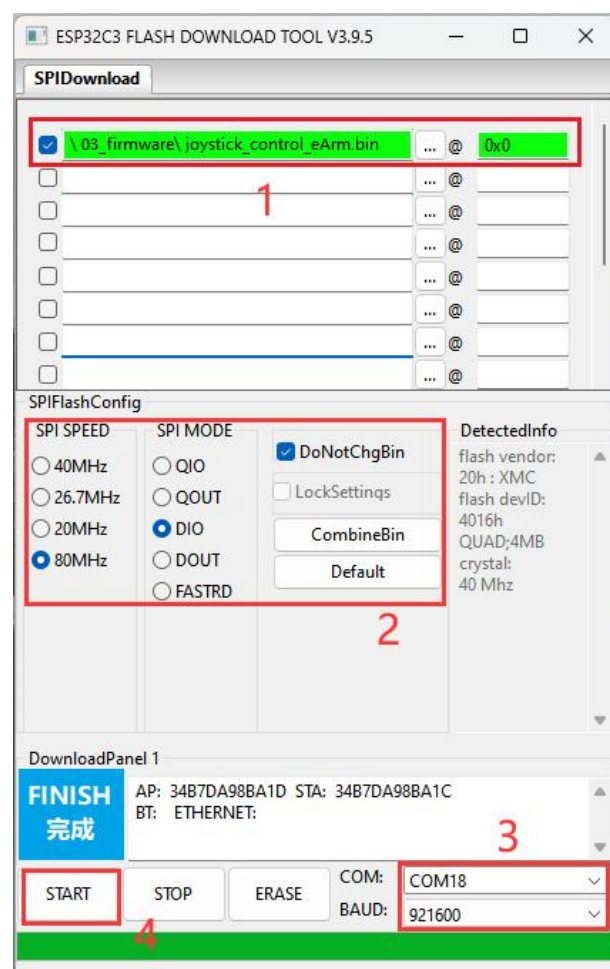


2.3.1 ジョイスティック制御ファームウェアの書き込み

USB ケーブルを使用して eArm を PC に接続します：書き込みツールを起動します：



書き込み対象のファームウェアを選択し、書き込み先アドレスを正しく入力します。



CH340 ドライバがインストールされていること、バッテリーが正しく接続されていること、および eArm の電源スイッチがオンになっている必要があります。これらを満たしていない場合、COM ポ

1: 書き込みオプションの 1 つ「」を選択し、「」をクリックして、チュートリアルで提供されている「**joystick_control_eArm.bin**」ファームウェアファイルを選択します。次に、「@」の後に書き込みアドレス (**0x0**) を正しく入力します。

ファイル名	書き込みアドレス
joystick_control_eArm.bin	0x0

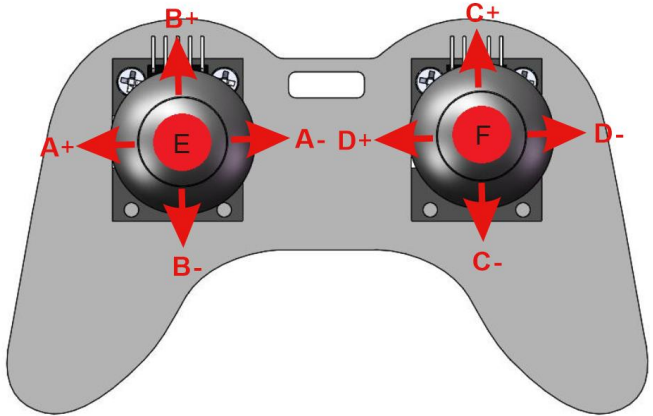
2：SPI SPEED で **80MHz**、SPI MODE で **DIO** を選択し、「」の「**DotNotChgBin**」にチェックを入れます。

3：コントロールボード用のシリアルポートを選択します。ここでは COM18 です（お使いのコンピュータでこのデバイスに実際に割り当てられているポートに基づいて選択してください）。ボーレートを **921600** に設定します。書き込みに失敗した場合は、ボーレートを **115200** などに下げてください。

4：クリックしてファームウェアの書き込みを開始します。

注記：ファームウェアの書き込みが完了したら、RST（リセット）ボタンを押すか、USB ケーブルを抜いてから、再度電源を入れてください。

ジョイスティック制御 - ロボットアーム操作ガイド：

	
A+：A servo を制御し、ロボットアームが左に回転します。	A-：A サーボを制御し、ロボットアームが右に回転します。
B+：B サーボを制御し、ロボットアームのメインアームが前方に振れます。	B-：B サーボを制御し、ロボットアームのメインアームが後方に振れます。
C+：C サーボを制御し、ロボットアームの細いアームが前方に振れる。	C-：C サーボを制御し、ロボットアームの細いアームを後方に振る。
D+：D サーボを制御し、ロボットアームのグリッパーが開く（緩む）。	D-：D サーボを制御し、ロボットアームのグリッパーを閉じ（締め付け）ます。
E：動作の記録 - E ボタンを 1 回押すと 1 つの動作を記録します（最大 20 個の動作を連続して記録可能）。押すたびに短いビープ音が鳴ります。記録容量の上限に達すると、ブザーから長いビープ音が 1 回鳴ります。	F：再生 - 動作シーケンスの記録が完了したら、F ボタンを 1 回押すと、記録された動作を連続で繰り返し再生（ループ再生）します。ブザーが 1 回鳴り、機械臂が記録された動作をループで繰り返します。 ループを終了するには、F ボタンを長押ししてください。ブザーが長く 1 回鳴り、再生が停止したことをお知らせします。

3

QA

3.1 USB シリアルポートが認識されない場合

- ☞ データ通信機能付き USB ケーブルを使用していますか？
- ☞ USB ケーブルは正しく接続されていますか？
- ☞ CH340 USB ドライバーはインストールされていますか？

3.2 ロボットアームが動作しない場合

- ☞ 取り付け時に電源スイッチを ON にしてサーボの角度を初期化しましたか？
- ☞ バッテリー残量は十分ですか？

3.3 その他の問題

- ☞ 組み立てが正しいか確認してください
- ☞ バッテリー残量が十分か確認してください
- ☞ 仕様通りのバッテリーを使用しているか確認してください

4

お問い合わせ

技術的な問題が発生した場合やフィードバックがありましたら、お気軽にご連絡ください

 support@siyeenove.com

 <https://siyeenove.com>